

Vorlesung Cybersicherheit, Sommersemester 2025

# Asymmetrische Kryptografie: Das RSA Kryptosystem

Prof. Dr.-Ing. Lucas Vincenzo Davi  
Fakultät für Informatik  
Arbeitsgruppe Systemsicherheit <https://www.syssec.wiwi.uni-due.de/>  
Universität Duisburg-Essen

© SYSSEC, Prof. Dr. Lucas Davi

# Rückblick und Themen heute

## Rückblick:

- Advanced Encryption Standard (AES)
- Betriebsmodi von symmetrischen Chiffren
- Einführung asymmetrische Kryptografie

## Themen heute:

- RSA Schlüsselerzeugung
- RSA Verschlüsselung mit Beispielrechnung
- Square-and-Multiply

# RSA

<http://people.csail.mit.edu/rivest/Rsapaper.pdf>

- Verschlüsselung
  - Öffentlicher Schlüssel:  $(n, e) = k_{pub}$
  - Verschlüsselungsfunktion:  $y = e_{k_{pub}}(x) \equiv x^e \mod n$  mit  $x, y \in \mathbb{Z}_n$
- Entschlüsselung
  - Privater Schlüssel:  $(d) = k_{pr}$
  - Entschlüsselungsfunktion:  $x = d_{k_{pr}}(y) \equiv y^d \mod n$  mit  $x, y \in \mathbb{Z}_n$

# Mathematische Intuition bei RSA

## (Beispiel Multiplikation in mod 14)

|    | 0 | 1   | 2  | 3   | 4  | 5   | 6  | 7 | 8  | 9   | 10 | 11  | 12 | 13  |
|----|---|-----|----|-----|----|-----|----|---|----|-----|----|-----|----|-----|
| 0  | 0 | 0   | 0  | 0   | 0  | 0   | 0  | 0 | 0  | 0   | 0  | 0   | 0  | 0   |
| 1  | 0 | *1* | 2  | 3   | 4  | 5   | 6  | 7 | 8  | 9   | 10 | 11  | 12 | 13  |
| 2  | 0 | 2   | 4  | 6   | 8  | 10  | 12 | 0 | 2  | 4   | 6  | 8   | 10 | 12  |
| 3  | 0 | 3   | 6  | 9   | 12 | *1* | 4  | 7 | 10 | 13  | 2  | 5   | 8  | 11  |
| 4  | 0 | 4   | 8  | 12  | 2  | 6   | 10 | 0 | 4  | 8   | 12 | 2   | 6  | 10  |
| 5  | 0 | 5   | 10 | *1* | 6  | 11  | 2  | 7 | 12 | 3   | 8  | 13  | 4  | 9   |
| 6  | 0 | 6   | 12 | 4   | 10 | 2   | 8  | 0 | 6  | 12  | 4  | 10  | 2  | 8   |
| 7  | 0 | 7   | 0  | 7   | 0  | 7   | 0  | 7 | 0  | 7   | 0  | 7   | 0  | 7   |
| 8  | 0 | 8   | 2  | 10  | 4  | 12  | 6  | 0 | 8  | 2   | 10 | 4   | 12 | 6   |
| 9  | 0 | 9   | 4  | 13  | 8  | 3   | 12 | 7 | 2  | 11  | 6  | *1* | 10 | 5   |
| 10 | 0 | 10  | 6  | 2   | 12 | 8   | 4  | 0 | 10 | 6   | 2  | 12  | 8  | 4   |
| 11 | 0 | 11  | 8  | 5   | 2  | 13  | 10 | 7 | 4  | *1* | 12 | 9   | 6  | 3   |
| 12 | 0 | 12  | 10 | 8   | 6  | 4   | 2  | 0 | 12 | 10  | 8  | 6   | 4  | 2   |
| 13 | 0 | 13  | 12 | 11  | 10 | 9   | 8  | 7 | 6  | 5   | 4  | 3   | 2  | *1* |

Tabelle erzeugt von ChatGPT

$$x^1 \cdot x^{-1} \bmod n = 1$$

$$5 \cdot 2^x \bmod 14 = 10^y$$

$$10 \cdot 3 \bmod 14 = 2(x)$$



1. Wähle zwei (große!) Primzahlen  $p$  und  $q$
2. Berechne:  $n = p \cdot q$
3. Berechne:  $\phi(n) = (p - 1)(q - 1)$
4. Wähle den öffentlichen Exponenten  $e \in \{1, 2, \dots, \phi(n) - 1\}$  mit  $\text{ggT}(e, \phi(n)) = 1$
5. Berechne den privaten Schlüssel  $d$  sodass  $d \cdot e = 1 \bmod \phi(n)$

Beachte:

- $d$  ist die Inverse von  $e$ , d.h.  $d \equiv e^{-1} \bmod \phi(n)$
- Schritt 5 könnte also auch so geschrieben werden:  $e^{-1} \cdot e = 1 \bmod \phi(n)$

# Schritt 1 der Schlüsselerzeugung

Wähle zwei (große!) Primzahlen  $p$  und  $q$

- Aufwändigster Schritt bei der Schlüsselgenerierung
- Finden zweier sehr großer Primzahlen erfordert das Verwenden von Primzahlentest Algorithmen, z.B. Fermatsche Primzahlentest, s. [https://de.wikipedia.org/wiki/Fermatscher\\_Primzahltest](https://de.wikipedia.org/wiki/Fermatscher_Primzahltest)
- Vereinfacht formulierte Strategie:
  - Nutze ein RNG (Random Number Generator) um eine ganze (z.B. 1024 Bit) Zahl zu wählen und auf Primzahl zu testen
  - Wiederhole den Vorgang bis eine Primzahl gefunden wurde

# Schritt 2 der Schlüsselerzeugung

$$\text{Berechne: } n = p \cdot q$$

- Einfacher Berechnungsschritt
- Wie groß wird  $n$  ?
  - Faustregel: Verdoppelung der Bitlänge
  - 1024 mal 1024 Bit ergibt also 2048 Bit Wert
- Faktorisierung von  $n$  unmöglich aufgrund der sehr großen Zahlenlänge



# Schritt 3 der Schlüsselerzeugung

$$\text{Berechne: } \phi(n) = (p - 1)(q - 1)$$

## Nutzen der Eulerschen Phi-Funktion:

- Diese Funktion gibt allgemein an wie viele teilerfremde Werte der Parameter  $n$  hat
- Beispiel:
  - $\phi(6) = 2$  weil nur 2 Zahlen, nämlich die Zahl 1 und 5 teilerfremd zu 6 sind, also  $\text{ggT}(1,6) = 1$  und  $\text{ggT}(5,6) = 1$
- Es gilt: Falls  $n$  eine Primzahl ist, dann ist  $\phi(n) = n - 1$ 
  - Beispiel:  $\phi(7) = 6$
- Ferner gilt: Falls 2 Zahlen (hier  $p, q$ ) Primzahlen sind, dann ist  $\phi(n) = \phi(p \cdot q) = \phi(p) \cdot \phi(q) = (p - 1)(q - 1)$

# Beispiele Eulersche Phi-Funktion

$$\phi(6) = 2$$

$$\text{SS}^+(0, 6) = 6$$

$$\text{SS}^+(1, 6) = 1 *$$

$$\text{SS}^+(2, 6) = 2$$

$$\text{SS}^+(3, 6) = 3$$

$$\text{SS}^+(4, 6) = 2$$

$$\text{SS}^+(5, 6) = 1 *$$

$$\phi(7) = 6 = 7 - 1$$

$$\text{SS}^+(0, 7) = 7$$

$$\text{SS}^+(1, 7) = 1 *$$

$$\text{SS}^+(2, 7) = 1 *$$

...

$$\text{SS}^+(6, 7) = 1 *$$

# Schritt 4 und 5 der Schlüsselerzeugung

Wähle den öffentlichen Exponenten  
 $e \in \{1, 2, \dots, \phi(n) - 1\}$  mit  $\text{ggT}(e, \phi(n)) = 1$

Berechne den privaten Schlüssel  $d$  sodass  $d \cdot e = 1 \bmod \phi(n)$

- Schritt 4 und 5 werden häufig zusammengefasst
  - Wähle zuerst einen Wert aus  $e \in \{1, 2, \dots, \phi(n) - 1\}$
  - Nutze dann den sogenannten Erweiterten Euklidischen Algorithmus (EEA)
  - EEA hat folgende Ausgabe:
    - $\text{ggT}(\phi(n), e) = s \cdot \phi(n) + t \cdot e$
  - Wenn der  $\text{ggT}(\phi(n), e) = 1$  ist, dann ist der öffentliche Schlüssel gültig
  - Der private Schlüssel  $d$  entspricht dem Parameter  $t$ , wobei  $t$  ist die Inverse von  $e$  modulo  $\phi(n)$  ist

# Der Erweiterte Euklidische Algorithmus (EEA)

- Eingabe: Zwei positive Zahlen  $r_0$  und  $r_1$  wobei  $r_0 > r_1$
- Ausgabe:  $ggT(r_0, r_1)$  sowie  $s$  und  $t$  mit  $ggT(r_0, r_1) = s \cdot r_0 + t \cdot r_1$
- Initialisierung:  $s_0 = 1, s_1 = 0, t_0 = 0, t_1 = 1, i = 1$
- Algorithmus:

**1. DO**

1.1  $i = i + 1$

1.2  $r_i = r_{i-2} \bmod r_{i-1}$

1.3  $q_{i-1} = (r_{i-2} - r_i) / r_{i-1}$

1.4  $s_i = s_{i-2} - q_{i-1} \cdot s_{i-1}$

1.5  $t_i = t_{i-2} - q_{i-1} \cdot t_{i-1}$

**WHILE**  $r_i \neq 0$

**2. RETURN**

$ggT(r_0, r_1) = r_{i-1}$

$s = s_{i-1}, t = t_{i-1}$

# Beispiel für RSA Berechnung

Alice

Bob

- $z_{\text{pub}}, z_{\text{pr}}$
1.  $p = 3, q = 11$
  2.  $n = p \cdot q = 33$
  3.  $\phi(n) = (p-1)(q-1)$   
 $= 2 \cdot 10$   
 $= 20$
  4. Wähle  $e = 3$
  5.  $d = 7$  (s. EEA)

Klartext

$\downarrow$   
 $x = 4$

$\leftarrow \begin{pmatrix} 33 & 3 \\ n & e \end{pmatrix}$

$$\begin{aligned} y &= x^e \bmod n \\ &= 4^3 \bmod 33 \\ &= 64 \bmod 33 \\ &= 31 \end{aligned}$$

$y = 31$

$$\begin{aligned} &\{ \dots -33, 0, 33 \dots \} \\ &\{ \dots -1, 32, \dots \} \\ &\{ \dots -2, 31, \dots \} \end{aligned}$$

$$\begin{aligned} y^d &= 31^7 \bmod 33 \\ &= -2^7 \bmod 33 \\ &= -128 \bmod 33 \\ &= 4 \bmod 33 \end{aligned}$$

# EEA für RSA Beispiel

$$\text{EEA: } r_0 = 20 \quad r_1 = 3$$

$$s_0 = 1, s_1 = 0, t_0 = 0, t_1 = 1, \hat{i} = 1$$

$$1.1: \hat{i} = \hat{i} + 1 = 2$$

$$1.2: r_2 = r_0 \bmod r_1$$

$$= 20 \bmod 3$$

$$= 2$$

$$1.3: q_1 = (r_0 - r_2) / r_1$$

$$= (20 - 2) / 3$$

$$= 6$$

$$1.4: s_2 = s_0 - q_1 \cdot s_1$$

$$= 1 - 6 \cdot 0$$

$$= 1$$

$$1.5: t_2 = t_0 - q_1 \cdot t_1$$

$$= 0 - 6 \cdot 1$$

$$= -6$$

$$1.1: \hat{i} = 3$$

$$1.2: r_3 = r_1 \bmod r_2$$

$$= 3 \bmod 2$$

$$= 1$$

$$1.3: q_2 = (r_1 - r_3) / r_2$$

$$= 3 - 1 / 2$$

$$= 1$$

$$1.4: s_3 = s_1 - q_2 \cdot s_2$$

$$= 0 - 1 \cdot 1$$

$$= -1$$

$$1.5: t_3 = t_1 - q_2 \cdot t_2$$

$$= 1 - 1 \cdot (-6)$$

$$= 7$$



# EEA für RSA Beispiel – Runde 3 und Ausgabe

$$1.1 \quad \bar{c} = 4$$

$$1.2 \quad r_4 = r_2 \bmod r_3 \\ = 2 \bmod 1 \\ = 0$$

$$1.3 \quad q_3 = 1$$

$$1.4 \quad s_4 = 2$$

$$1.5 \quad t_4 = -13$$

---

$$2. \text{ggT}(20, 3) = r_3 = 1 \quad \checkmark$$
$$s = s_3 = -1 \quad t = t_3 = 7$$

$$\text{ggT}(20, 3) = s \cdot r_0 + t \cdot r_1 \\ = -1 \cdot 20 + 7 \cdot 3 \\ = 1$$

$\downarrow$   
 $\mathbb{Z}_{pr}$

- Schnelle Verfahren für die Exponentiation von sehr großen Zahlen notwendig
  - Die naive Exponentiation von  $2^{1024}$  durch einfaches Multiplizieren benötigt ca.  $10^{300}$  Multiplikationen
- Deswegen wurden Algorithmen zur schnellen Exponentiation eingeführt
  - Der [Square-and-Multiply Algorithmus](#) für  $2^{1024}$  benötigt durchschnittlich nur  $1,5 \cdot 1024$  Quadrierungen und Multiplikationen
  - Der Algorithmus verwendet auf Basis der Binärdarstellung des Exponenten Quadrierung (Square) und Multiplikation (Multiplikation) und reduziert deutlich die Rechenkomplexität
  - Detailliertes Vorgehen:
    - Exponent in Binärformat konvertieren und Exponent vom Most Significant Bit (MSB) bis zum Least Significant Bit (LSB) abarbeiten
    - Beim MSB die Basis einfach übertragen; bei allen anderen Bits:
      - Beim Bitwert 0 des Exponenten wird nur eine Quadrierung ausgeführt
      - Beim Bitwert 1 des Exponenten wird sowohl quadriert als auch mit der Basis multipliziert

# Motivation für Square-and-Multiply Algorithmus

$$x^8: x \xrightarrow{SQ} x^2 \xrightarrow{MUL} x^3 \xrightarrow{MUL} x^4 \xrightarrow{MUL} x^5 \xrightarrow{MUL} x^6 \xrightarrow{MUL} x^7 \xrightarrow{MUL} x^8$$

$$x^8: x \xrightarrow{SQ} x^2 \xrightarrow{SQ} x^4 \xrightarrow{SQ} x^8 \quad \underline{1000}$$

$$x^{26}: x \xrightarrow{SQ} x^2 \xrightarrow{MUL} x^3 \xrightarrow{SQ} x^6 \xrightarrow{SQ} x^{12} \xrightarrow{MUL} x^{13} \xrightarrow{SQ} x^{26}$$

$$2 \times MUL, 4 \times SQ$$

$$x^{26}: x \overset{b_4}{\downarrow} 11010_b$$

$$b_4 = 1, b_3 = 1, b_2 = 0, b_1 = 1, b_0 = 0$$

$$b_4 (MSB) = 1: x$$

$$b_3 = 1: \begin{array}{l} SQ: x^2 \\ MUL: x^3 (x^2 \cdot x) \end{array}$$

$$b_2 = 0: SQ: x^6$$

$$b_1 = 1: \begin{array}{l} SQ: x^{12} \\ MUL: x^{13} \end{array}$$

$$b_0 = 0: SQ: x^{26}$$

# Rechenbeispiel Square-and-Multiply Algorithmus

Quelle: [https://www.youtube.com/watch?v=cbGB\\_V8MNk](https://www.youtube.com/watch?v=cbGB_V8MNk)

$$3^{45} \bmod 7 = \overset{b_5 \rightarrow 101101_b}{\underset{(x)}{3}} = 6 \bmod 7$$

$$b_5 = 1 : 3$$

$$b_4 = 0 : SQ \quad 3^2 = 9 \bmod 7 \\ = 2$$

$$b_3 = 1 : SQ : 2^2 \bmod 7 = 4 \\ MUL : 4 \cdot 3 \bmod 7 \equiv 5 \bmod 7$$

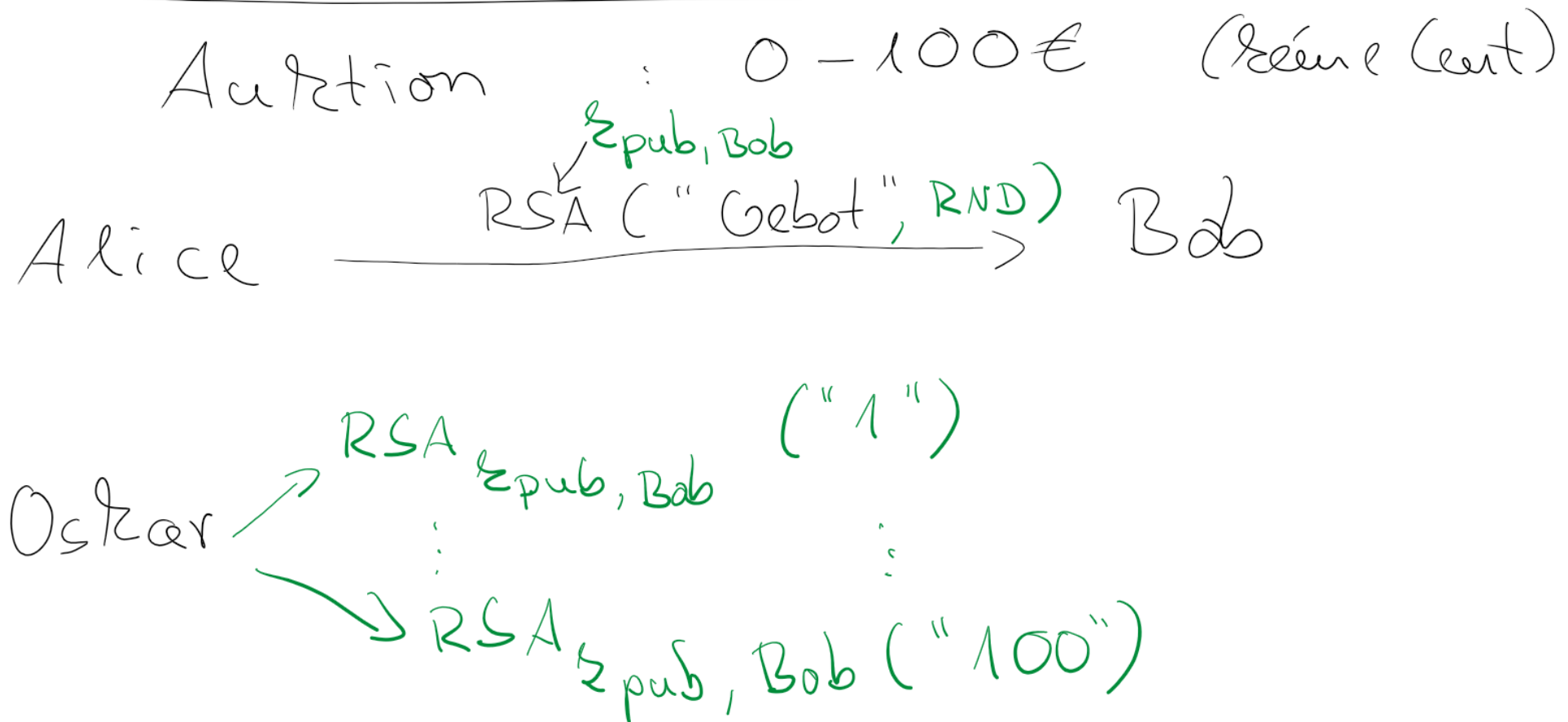
$$b_2 = 1 : SQ : 5^2 \bmod 7 \equiv 4 \bmod 7 \\ MUL : 4 \cdot 3 \bmod 7 \equiv 5 \bmod 7$$

$$b_1 = 0 : SQ : 5^2 \equiv 4 \bmod 7$$

$$b_0 = 1 : SQ : 4^2 \equiv 2 \bmod 7 \\ MUL : 2 \cdot 3 \bmod 7 = 6 \bmod 7$$

- RSA Probleme (Textbook-RSA)
  - Berechnungen sind deterministisch – gleicher Klartext führt immer zum selben Chifftrat
  - Angreifer kann durch Manipulation am Chifftrat Klartext Nachrichten manipulieren (s. nächste Folie)
- RSA mit Padding löst diese Probleme
  - Padding fügt zufällige Daten in den Klartext ein
- Weitere mögliche Angriffe auf RSA
  - Schnellere Faktorisierungsalgorithmen?
    - Es gab tatsächlich Fortschritte aber immer noch unmöglich für sehr große Zahlen
    - RSA-129 <https://www.youtube.com/watch?v=YQw124CtvO0>
  - Seitenkanalangriffe?
    - Stromverbrauch kann bei Square-and-Multiply auf den privaten Schlüssel schließen
- Effizienz
  - AES ist ca. 100-1000 mal schneller als RSA

# Deterministische Verschlüsselung bei RSA



# Beispiel: Textbook-RSA Angriff

- Angriffsszenario
  - Angreifer Oskar sieht das Chifftrat auf dem Kanal
  - Oskar wählt einen Parameter gleich 2 und verschlüsselt diesen mit dem öffentlichen Exponenten und multipliziert das Ergebnis mit dem Chifftrat
  - Die Entschlüsselung führt zum Klartext aber multipliziert mal 2

$$\begin{array}{c} Y \\ (s^e \cdot Y)^d = s^{\overset{\uparrow}{e} \overset{\uparrow}{d}} \cdot x^{\overset{\uparrow}{e} \overset{\uparrow}{d}} = s \cdot x \bmod n \\ \uparrow \\ x^e \end{array}$$



Vorlesung Cybersicherheit, Sommersemester 2025

# Vielen Dank für Ihre Aufmerksamkeit

Prof. Dr.-Ing. Lucas Vincenzo Davi  
Fakultät für Informatik  
Arbeitsgruppe Systemsicherheit  
Universität Duisburg-Essen

© SYSSEC, Prof. Dr. Lucas Davi